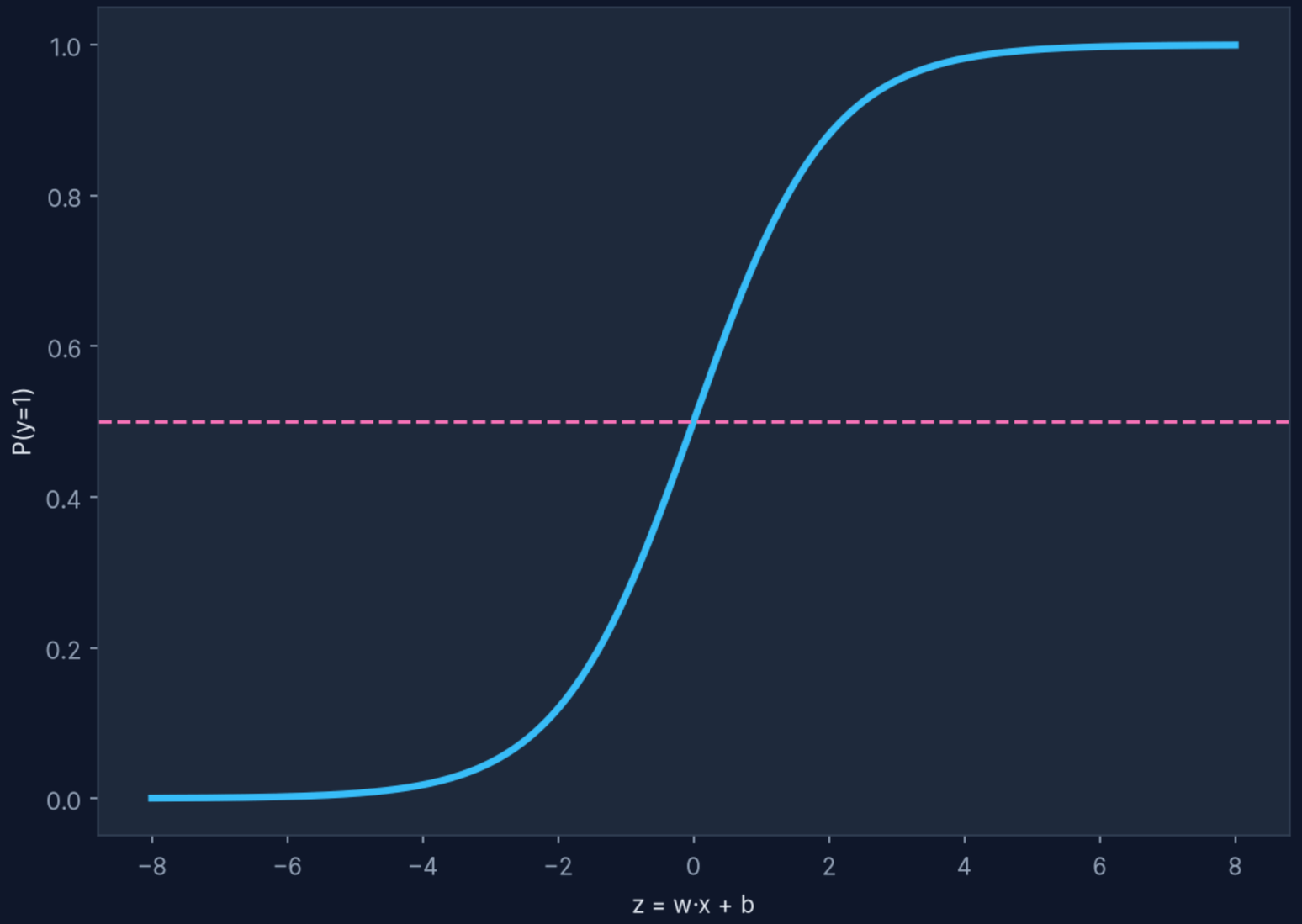
Logistic Regression

Linear models for classification

The problem

- Predict a class, e.g. spam vs not spam
- Want a calibrated probability, not just a label
- Linear regression outputs can be < 0 or > 1

Intuition: squash with a sigmoid



$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Maps any score to (0, 1)

Boundary where z = 0

Loss: cross-entropy

$$L = -\frac{1}{n}\sum_i [y_i \log p_i + (1 - y_i)\log(1 - p_i)]$$

$$\nabla_w L = \frac{1}{n} X^\top (p - y)$$

- Convex → gradient descent finds the global optimum

Binary training loop

```
def fit(self, X, y):
    X, y = np.asarray(X, float), np.asarray(y, float)
    n, d = X.shape
    self.w, self.b = np.zeros(d), 0.0
    for _ in range(self.epochs):
        p = sigmoid(X @ self.w + self.b)
        self.w -= self.lr * (X.T @ (p - y) / n + self.l2 * self.w)
        self.b -= self.lr * np.mean(p - y)
    return self
```

Multi-class: softmax

$$P(y = k) = \frac{e^{z_k}}{\sum_j e^{z_j}}$$

- One weight vector per class
- Subtract max(z) for numerical stability
- Same gradient form: $X^T(P - Y)$

Softmax regression

```
def softmax(z):
    z = z - z.max(axis=1, keepdims=True)
    e = np.exp(z)
    return e / e.sum(axis=1, keepdims=True)

def fit(self, X, y):
    X, y = np.asarray(X, float), np.asarray(y)
    self.classes_ = np.unique(y)
    Y = (y[:, None] == self.classes_[None, :]).astype(float)
    n, d = X.shape
    self.W, self.b = np.zeros((d, len(self.classes_))), np.zeros(len(self.classes_))
    for _ in range(self.epochs):
        P = softmax(X @ self.W + self.b)
        self.W -= self.lr * X.T @ (P - Y) / n
        self.b -= self.lr * (P - Y).mean(axis=0)
    return self
```

Complexity & trade-offs

- Training: O(n·d·k) per epoch; prediction O(d·k)
- Interpretable weights (log-odds)
- Only linear boundaries unless you engineer features
- Tune the threshold for precision vs recall

Recap

- Sigmoid / softmax turn scores into probabilities
- Minimise cross-entropy
- Gradient = $X^T(p - y)$
- Strong, fast, interpretable baseline